AIハーネスの実行予算制御と総費用評価

仕事の品質を保ち、不要な処理を減らす

AIに任せる仕事には、調査・判断・実行・確認が含まれる。



情報を集める

文書・要求・観測

処理して変更する

コード・API・ファイル

完了を確かめる

要件と最終状態

ハーネスは、渡す情報・ツール実行・状態・検証・停止を制御する。
減らしたいのは、不要な取得・共有・追加生成・やり直しの総費用。

上限で止めても、仕事の完了は保証できない

実行器がしていたこと

API・token・時間の上限を監視。
上限に届いたら処理を止める。

AIに伝えていなかったこと

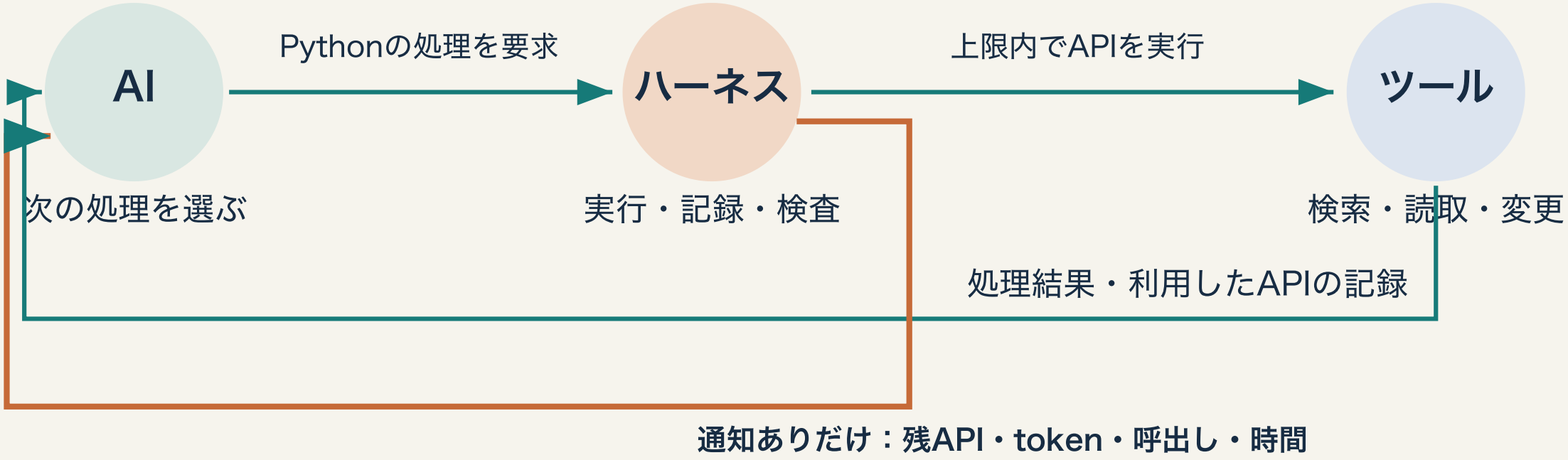
あと何回APIを使えるか。
あとどれだけ生成・実行できるか。

過去15条件・102入力を点検し、明示的な残予算通知がないことを確認。

> 仮説：残量を知れば、取得・実行・確認の配分が変わる可能性がある。

今回の実装：実行記録から残予算を毎回知らせる

同じ課題・同じ作業状態・同じ上限



実験の前提：同じ途中状態から、通知だけを変える

AppWorldの模擬アプリ：写真整理／電気代の分担請求／返金／曲情報の書出。

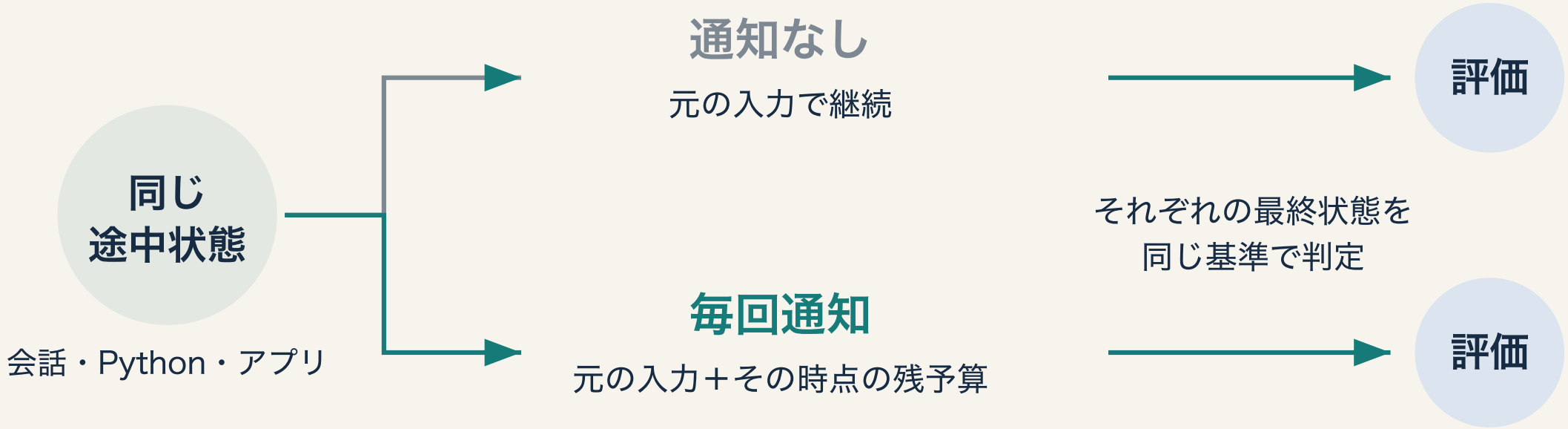
条件	AIが受け取る入力
通知なし	全履歴・現在の実行結果
毎回通知	同じ情報 ＋ その時点の残予算

８応答後の会話・Python・アプリ状態を復元。

各条件：16生成・12万token・480秒。APIは元の上限で前半も計上。

MiniMax M3 IQ4_XS。４課題×２条件×各１回。token・API・時間は別の費用として記録。

手順：開始状態をそろえ、継続後の成否と費用を測る



８応答後の状態を各条件へ復元。評価結果は実行中のAIに返さない。

終了状況：評価済み６件、途中終了１件、未実行１件

6

最終評価まで終了

３課題の２条件。
使用量も確定。

1

時間上限で終了

書出・通知あり。
最終生成の使用量不明。

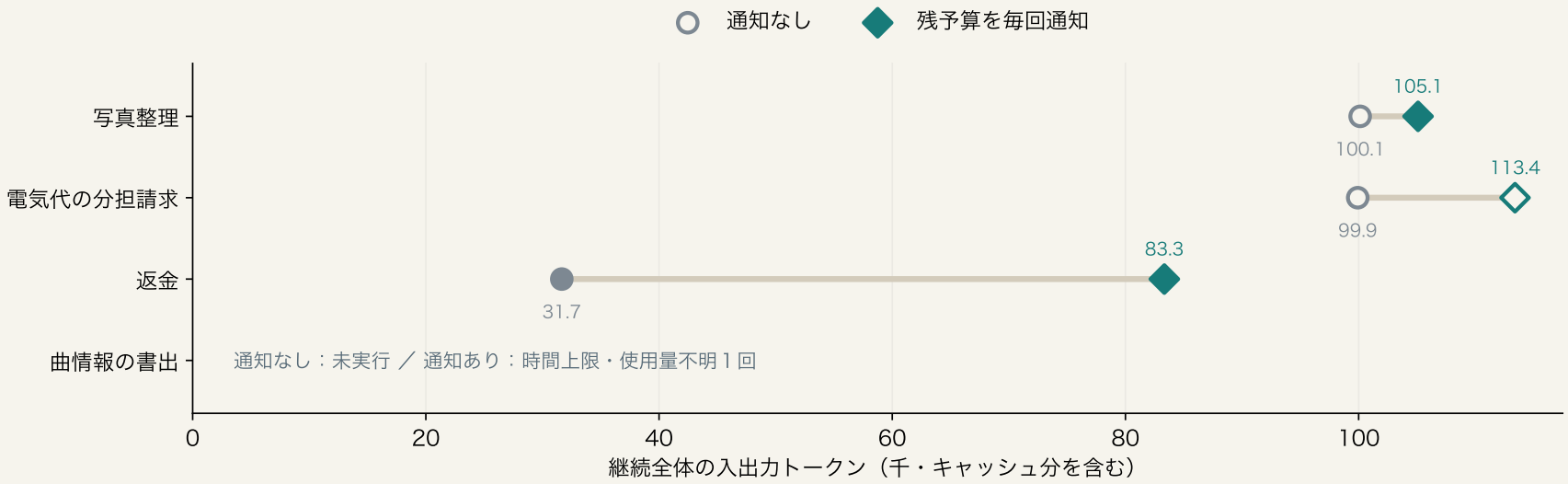
1

規則により未実行

書出・通知なし。
置換試行はしない。

> 予定した４課題の比較は未完遂。欠測を失敗０点や使用量０に置き換えない。

評価済み３課題：成否と費用を対応させる



塗り：成功／白抜き：未達。共通前半・実験用の復元は別計上。未達の安さは効率化としない。

条件	成功	継続token	AI呼出し	後続API	継続秒
通知なし	1/3	231,709	19	106	679.3
残予算を毎回通知	2/3	301,840	21	147	885.5

課題ごとの対応：成功と打切りを混ぜて評価しない

課題	通知なし	毎回通知	通知あり−なしのtoken
写真整理	未達	成功	+4,966
電気代の分担請求	未達	未達	+13,488
返金	成功	成功	+51,677
曲情報の書出	未実行	途中終了・未評価	比較不可

> 同じ成功件数だけでは、同じ課題を解けたことや一般的な品質維持を示せない。

考察：残予算と、呼出しを止める規則を分ける

実行器は「次の入力＋最大4,096出力token」を予約していた。

予約上限で停止した条件	残token	次の入力	出力に残せる量
写真整理・通知なし	19,871	17,400	2,471
分担請求・通知なし	20,078	16,200	3,878
分担請求・毎回通知	6,590	16,123	0

終了後に入力長だけを再計数。生成・環境操作は追加していない。

> ２条件には短い出力枠が残る。ただし、それで仕事を完了できるかは未検証。

新規性の現在地：今回整えたのは、比較の土台

候補・部品	現在の判断
一律の履歴省略	過去の比較で総tokenが増え、採用を見送った。
必要項目のキャッシュ	実際の利用確認で再利用０。拡大を見送った。
残予算の通知	Budget Tracker等と重なる既存の対照方式。
独自方式と性能維持	現在の証拠だけでは確立していない。

次に必要なこと：通常の対処を超える改善を絞る

研究として確かめる

どの取得・反復が増えているか。
予算通知と停止規則の効果を分ける。
残る失敗に必要な制御を絞る。

レターへ向けた現状

10月31日の投稿が目標。
中心となる独自の主張が不足。
４課題の結果を一般化しない。

> 目標は変わらない：必要な仕事を完了し、不要な取得・共有・実行を減らす。